

FULL STACK DEVELOPMENT

FASE 4 | AULA 01 -

ARQUITETURA MICROFRONTEND

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

Vamos começar entendendo o contexto: com o aumento da popularidade dos microservices, muitas pessoas desenvolvedoras perceberam que a arquitetura monolítica, tanto no backend quanto no frontend, pode trazer algumas limitações. Por exemplo, você já deve ter enfrentado a dor de cabeça de tentar integrar novas funcionalidades em uma aplicação gigante, ou de lidar com ferramentas de build que parecem lutar contra você a cada nova atualização. E nem vamos falar da dificuldade em escalar uma aplicação monolítica com várias equipes trabalhando em paralelo sem pisar no trabalho do outro.

Então, como podemos resolver esses problemas? A resposta está em fragmentar o frontend em pedaços menores, independentes, mas que ainda funcionam juntos, como um todo coeso para o usuário final. É como montar um quebra-cabeça onde cada peça é uma parte do frontend, mas todas se encaixam perfeitamente para formar a imagem completa.

E não é só isso! Os microfrontends não apenas tornam o código mais manutenível e as equipes mais autônomas, mas também permitem atualizações incrementais. Isso significa que você pode lançar novas funcionalidades ou correções de bugs sem precisar reconstruir toda a aplicação. Mas como tudo na vida, essa arquitetura também tem seus desafios, como a possível duplicação de dependências e a necessidade de gerenciar bem os riscos e benefícios.

Vamos discutir tudo isso em detalhes: desde como manter a coesão visual entre diferentes microfrontends até as estratégias de comunicação entre eles, garantindo que a experiência do usuário seja fluida e sem falhas. E claro, mostraremos como as equipes podem colaborar de forma eficaz, evitando acoplamentos acidentais e promovendo práticas de qualidade que tornam o código mais limpo e fácil de manter.

Essa aula é uma introdução, mas cheia de insights sobre como os microfrontends podem revolucionar o desenvolvimento frontend. Estamos prestes a explorar uma abordagem que não só resolve problemas comuns, mas também abre novas possibilidades para escalar e evoluir suas aplicações de maneira eficiente e modular.

HANDS ON

Nesta aula, mergulharemos nos aspectos teóricos que sustentam a arquitetura de microfrontends. Embora não tenhamos uma atividade prática imediata, este é um momento crucial para consolidar os conceitos que discutimos. Compreender a teoria por trás dos microfrontends é essencial para aplicá-los com sucesso em projetos reais, garantindo que você possa tomar decisões informadas sobre quando e como fragmentar suas aplicações frontend.

Nas videoaulas, vamos detalhar as bases teóricas dessa arquitetura, explicando como e por qual motivo os microfrontends podem ser a solução ideal para problemas de escalabilidade e manutenção que muitos especialistas em desenvolvimento enfrentam ao trabalhar com monólitos frontend. Você verá como a teoria se conecta diretamente com a prática, preparando você para aplicar esses conceitos em seu trabalho diário.

Ao final da aula, você estará bem equipado(a) para entender não apenas o “como”, mas também o “porquê” dos microfrontends, e estará pronto(a) para explorar mais profundamente a aplicação desses conceitos nas próximas aulas. Vamos juntos?

SAIBA MAIS

Monorepos e Multi Repos

Vamos falar sobre uma questão que pode parecer simples à primeira vista, mas que tem implicações profundas no desenvolvimento de software: a escolha entre monorepos e multi repos. Essas duas abordagens têm a ver com como organizamos o código de um projeto, e cada uma tem peculiaridades que podem impactar o trabalho da equipe, a gestão de dependências, a maneira como os deploys são feitos, e até mesmo o gerenciamento de versões.

Vamos começar entendendo o que é um monorepo. Imagine que você tem um grande projeto que envolve várias partes diferentes: talvez um frontend, um backend, algumas bibliotecas compartilhadas, scripts de automação, e assim por diante. Num monorepo, todo esse código está em um único repositório. Isso significa que você tem uma única fonte da verdade, onde todas as partes do sistema são versionadas juntas. A grande vantagem aqui é a consistência: se você precisar atualizar uma dependência que afeta tanto o frontend quanto o backend, você pode fazer isso em um único commit. Também fica mais fácil compartilhar código entre diferentes partes do projeto, já que tudo está ali, no mesmo lugar.

Por outro lado, os monorepos podem se tornar bastante complexos, especialmente à medida que o projeto cresce. Imagine um repositório gigantesco com centenas de milhares de linhas de código. Ferramentas como builds e pipelines de CI/CD podem começar a ficar lentas, e gerir permissões de acesso pode ser complicado. É por isso que grandes empresas que adotam monorepos, como Google e Facebook, investem em ferramentas avançadas como NX ou Turborepo para gerenciar essa complexidade.

Agora, vamos falar sobre multi repos, que é a abordagem mais tradicional. Aqui, cada projeto ou serviço tem seu próprio repositório. Se você tem um frontend e um backend, eles vivem em repositórios separados. Isso facilita a escalabilidade e permite que as equipes trabalhem de forma mais independente. Cada repositório é mais fácil de gerenciar, e os builds costumam ser mais rápidos, porque você está lidando com menos código de uma vez. Além disso, você tem mais controle sobre as permissões:

se uma equipe só precisa trabalhar no frontend, você pode dar acesso para que ela trabalhe apenas ao repositório do frontend.

No entanto, os multi repos trazem seus próprios desafios. Compartilhar código entre diferentes repositórios pode ser mais complicado, pois você precisa versionar e publicar pacotes ou bibliotecas, o que adiciona uma camada extra de complexidade. Manter a consistência entre repositórios — como garantir que todos estão usando as mesmas versões de ferramentas ou dependências — também pode ser um problema.

Então, como decidir qual abordagem usar? Depende muito do tamanho da sua equipe, da natureza do seu projeto e das ferramentas que você tem à disposição. Se você está em uma startup com uma equipe pequena, um multi repo pode ser mais prático. Mas se você está em uma grande empresa onde diferentes partes do sistema precisam interagir muito de perto, um monorepo pode fazer mais sentido.

No final das contas, tanto os monorepos quanto os multi repos têm seu lugar no desenvolvimento moderno. A chave é entender os trade-offs e escolher a abordagem que melhor atende às necessidades do seu projeto e da sua equipe. E lembre-se, não importa qual seja a estrutura que você escolha, ferramentas como GitHub Actions podem ajudar a automatizar seus processos de build e deploy, tornando seu fluxo de trabalho mais eficiente, seja qual for o modelo de repositório que você adote.

Essas decisões de arquitetura são fundamentais, e espero que agora você tenha uma visão mais clara sobre como os monorepos e multi repos podem impactar o desenvolvimento do seu projeto. Se você quiser se aprofundar ainda mais, recomendamos conferir as documentações e exemplos de uso que grandes empresas como Google e Facebook disponibilizam sobre suas experiências com essas abordagens.

Agora que já discutimos os prós e contras de monorepos e multi repos, é importante considerar como essas abordagens se aplicam ao contexto de microfrontends. Quando falamos em microfrontends, a fragmentação do frontend em pedaços menores e independentes parece combinar bem com a abordagem de multi repos. Cada microfrontend pode ser desenvolvido, versionado e implantado de forma autônoma, o que reforça a ideia de equipes independentes trabalhando em paralelo. Com multi repos, você consegue isolar completamente cada microfrontend, o que

pode ser uma grande vantagem em termos de gerenciamento de dependências e segurança.

Por outro lado, um monorepo pode facilitar a integração e a comunicação entre os diferentes microfrontends. Em um monorepo, você tem uma visão completa de todos os microfrontends e do código compartilhado, o que pode ser útil para manter a coesão visual e evitar a duplicação de componentes ou estilos. Além disso, ao versionar todos os microfrontends juntos, você pode garantir que as mudanças sejam implementadas de maneira sincronizada, reduzindo o risco de conflitos ou problemas de compatibilidade.

Ao decidir entre monorepo e multi repos para um projeto com microfrontends, você precisa pesar a necessidade de autonomia e isolamento das equipes contra a facilidade de integração e coesão do código. A escolha vai depender muito do tamanho do seu projeto, da complexidade dos microfrontends e da estrutura da sua equipe.

Um monolito modularizado tem o seu valor

Quando se fala em arquitetura de frontend, os microfrontends vêm ganhando muita atenção por sua promessa de modularidade, escalabilidade e autonomia das equipes. No entanto, é importante lembrar que, como em qualquer abordagem arquitetural, os microfrontends não são uma bala de prata. Eles resolvem muitos problemas, mas também trazem novos desafios e complexidades. Em muitos casos, trabalhar com um monolito modularizado pode ser uma alternativa viável e até preferível.

Vamos pensar no conceito de um monolito modularizado. Em vez de fragmentar o frontend em microfrontends totalmente independentes, você mantém uma única aplicação, mas organiza o código de forma modular. Isso significa que cada parte do sistema é bem definida e encapsulada dentro do monólito, mas ainda faz parte de um todo coeso. Essa abordagem pode oferecer um equilíbrio interessante entre a simplicidade de um monolito e os benefícios de modularidade que atraem as pessoas para os microfrontends.

Um dos grandes benefícios de um monolito modularizado é a simplicidade. Como tudo está em um único repositório e numa única aplicação, você evita muitos dos desafios que os microfrontends trazem, como a necessidade de gerenciar

múltiplos pipelines de CI/CD, resolver problemas de comunicação entre diferentes microfrontends, e lidar com a duplicação de dependências. Em um monolito modularizado, a comunicação entre diferentes módulos é mais direta, e a coesão visual e funcional é mais fácil de manter, já que todos estão trabalhando na mesma base de código.

Além disso, o acoplamento acidental pode ser menos problemático em um monolito modularizado. Em microfrontends, o risco de criar dependências ocultas entre os diferentes pedaços da aplicação é real, e pode levar a uma complexidade inesperada. No monolito modularizado, como todos os módulos estão no mesmo lugar, é mais fácil identificar e gerenciar essas dependências.

Por outro lado, os microfrontends são muitas vezes escolhidos para permitir que diferentes equipes trabalhem de forma completamente independente, sem precisar se preocupar com as partes da aplicação que não estão sob sua responsabilidade direta. Mas essa independência vem com o custo de complexidade adicional, tanto em termos de infraestrutura quanto de coordenação entre equipes. Por exemplo, garantir que o estilo visual de diferentes microfrontends seja consistente, ou que a experiência do usuário seja fluida e unificada, pode exigir um esforço considerável.

Adoção de microfrontends

Optar por uma arquitetura de microfrontends é uma decisão que pode trazer muitos benefícios, mas que também exige uma série de cuidados para garantir que esses benefícios sejam realmente alcançados. Enquanto a ideia de dividir grandes aplicações em pedaços menores e independentes é atraente, essa abordagem não vem sem seus desafios e armadilhas.

Um dos primeiros cuidados a ter em mente é a complexidade de infraestrutura. Ao fragmentar sua aplicação em múltiplos microfrontends, você está, na prática, multiplicando a quantidade de sistemas que precisam ser gerenciados, testados e implantados. Isso significa mais pipelines de CI/CD para configurar, mais repositórios para gerenciar, e mais componentes para monitorar e manter. O aumento na carga de trabalho de manutenção pode ser significativo, especialmente para equipes menores ou menos experientes. Cada microfrontend precisa ser tratado como uma

miniaplicação por si só, com seus próprios processos de build, deploy e monitoramento, o que pode rapidamente se tornar uma sobrecarga.

Além disso, a coerência visual e funcional entre os microfrontends pode ser um desafio. Com equipes diferentes trabalhando em partes distintas da aplicação, há o risco de que cada microfrontend acabe desenvolvendo seu próprio estilo e comportamento, o que pode resultar em uma experiência de usuário inconsistente. Para mitigar esse risco, é crucial ter um design system sólido e acordos claros sobre como os componentes visuais e comportamentais devem ser implementados. No entanto, mesmo com esses cuidados, garantir a uniformidade pode exigir uma coordenação significativa entre as equipes.

Outro ponto importante é a gestão de dependências. Cada microfrontend pode ter suas próprias dependências, e isso pode levar à duplicação de pacotes ou mesmo a conflitos entre diferentes versões de uma mesma biblioteca. Embora os microfrontends ofereçam a liberdade de escolher diferentes tecnologias ou versões para cada parte da aplicação, essa liberdade precisa ser exercida com cautela para evitar um aumento na complexidade do projeto. Ferramentas como monorepos podem ajudar a centralizar e gerenciar essas dependências, mas, novamente, isso adiciona uma camada extra de complexidade.

Também devemos considerar a comunicação entre os microfrontends. Como cada microfrontend é uma unidade independente, a comunicação entre eles deve ser bem definida e controlada para evitar acoplamentos indesejados. Dependendo da arquitetura, essa comunicação pode ser feita via eventos, chamadas API, ou até mesmo compartilhamento de estado global. Independentemente do método escolhido, é vital garantir que os contratos de comunicação sejam claros e estáveis para evitar que mudanças em um microfrontend quebrem outros.

Por fim, há o desafio da coordenação e governança. Em uma arquitetura de microfrontends, a responsabilidade é distribuída, o que é ótimo para autonomia das equipes, mas pode complicar a governança do projeto como um todo. É necessário ter políticas claras sobre como as decisões são tomadas e como as mudanças são implementadas para garantir que o projeto mantenha uma direção coesa e eficiente.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, mergulhamos nos fundamentos teóricos da arquitetura de microfrontends, uma abordagem cada vez mais popular no desenvolvimento de aplicações web. Exploramos como a fragmentação de uma aplicação monolítica em pedaços menores e independentes pode resolver muitos dos desafios enfrentados por equipes de desenvolvimento, como a escalabilidade, a complexidade de integração de novas funcionalidades e a dificuldade em manter coesão em grandes projetos.

Começamos discutindo o contexto que levou à popularidade dos microfrontends, especialmente com o surgimento dos microservices no backend. Você viu como os monólitos, tanto no backend quanto no frontend, podem limitar a flexibilidade das equipes, dificultando a adoção de novas ferramentas e complicando o trabalho em equipe. A solução? Dividir a aplicação em partes menores, mas mantendo a experiência do usuário final coesa e integrada.

Falamos sobre os benefícios dessa abordagem, como a autonomia das equipes, a facilidade em implementar atualizações incrementais e a redução da complexidade por meio da definição clara de limites entre os contextos da aplicação. No entanto, também discutimos os desafios, como a possível duplicação de dependências, a necessidade de uma governança eficaz e os riscos de fragmentação no modo de trabalho das equipes.

Exploramos ainda a importância de um design system sólido para garantir a consistência visual entre os microfrontends e a necessidade de estratégias claras para a comunicação entre eles, seja por meio de eventos personalizados, callbacks ou até mesmo utilizando a barra de endereços como mecanismo de comunicação.

Por fim, discutimos a importância de uma boa gestão de riscos e a avaliação cuidadosa dos benefícios ao optar por microfrontends, lembrando sempre que essa não é uma solução universal e que a escolha dessa arquitetura deve ser bem fundamentada.

REFERÊNCIAS

JACKSON, C. **Micro Frontends**. 2019. Disponível em: <https://martinfowler.com/articles/micro-frontends.html> . Acesso em: 14 ago. 2024.

MICRO Frontends Documentation. **Micro Frontends Overview**. Disponível em: <https://micro-frontends.org/> . Acesso em: 14 ago. 2024.

SINGLE-SPA Documentation. **Microfrontends Concept**. Disponível em: <https://single-spa.js.org/docs/microfrontends-concept/>. Acesso em: 14 ago. 2024.

EXEMPLO

PALAVRAS-CHAVE

Modularidade. Arquitetura Frontend. Microservices.

EMSE

POS TECH